

Privacy in Publish Subscribe Systems

Jacob Olinger
Portland State University
Portland, Oregon
jacoboli@pdx.edu

Primal Pappachan
Portland State University
Portland, Oregon
primal@pdx.edu

ABSTRACT

ACM Reference Format:

Jacob Olinger and Primal Pappachan. 2026. Privacy in Publish Subscribe Systems. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Every ten minutes, a traffic sensor in Rome reports how many vehicles occupy a road segment. Every second, a smart meter in a residential building transmits instantaneous power draw. Every heartbeat, a wearable health monitor pushes a cardiac event to a cloud dashboard. These are instances of the same architectural pattern: publish–subscribe messaging, in which publishers emit streams of events and subscribers consume them through a broker that matches, routes, and, as we shall argue, must also protect them.

The privacy stakes are concrete. A subscriber who receives traffic counts can re-identify the commuter behind a sequence of road-segment events [1]. A subscriber who receives smart-meter readings can infer whether a household is occupied. A subscriber who receives cardiac alerts can deduce the health condition of a patient. In each case, the harm arises from two distinct channels: the adversary may learn who generated a message (publisher privacy), or what the message contains (content privacy). These channels are orthogonal, protecting one does not protect the other, and each demands its own formal guarantee.

We formalize this setting, show how classical privacy notions adapt to it, and surface a structural question that prior work has not addressed: where in the publish–subscribe pipeline should privacy be enforced, and what does that choice imply for the guarantee?

We make the following research in this paper:

(1)

2 RELATED WORK

<https://www.vldb.org/2014/program/http://www.vldb.org/pvldb/vol7/p1155kellaris.pdf> <https://www.vldb.org/pvldb/vol18/p5652-shaowang.pdf>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2026 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

3 SYSTEM MODEL

We consider a publish–subscribe system consisting of publishers $\mathcal{P}$, a broker, and subscribers. Each message is a tuple

$$m = (p, x, s, \tau) \in \mathcal{P} \times \mathcal{X} \times \mathcal{S} \times \mathcal{T},$$

comprising publisher identity p , payload x , topic s , and timestamp τ . The broker transforms each message via a transformation $T: \mathcal{M} \rightarrow \mathcal{Y}$ before delivery; subscribers observe only the rewritten output $y = T(m)$.

The message stream is unbounded: messages $m_1, m_2, m_3, \dots$ arrive indefinitely, and every message produces a corresponding output $y_i \in \mathcal{M}$; the broker does not aggregate messages into batches or suppress delivery.

Typical differential privacy guarantees [?] are defined over aggregates of messages. Applying such mechanisms to streaming settings requires buffering or batching outputs, as in prior work on live message streams [1], which conflicts with the per-message delivery semantics of publish–subscribe architectures.

Instead, to reason about privacy, we use a rolling window of size w that slides over the stream one message at a time, applying a privacy guarantee to each window $W_t = \{m_t, m_{t+1}, \dots, m_{t+w-1}\}$.

Definition 3.1 (Rolling Window). For a positive integer w , the rolling window at position i is $R_i = (m_{i-w+1}, \dots, m_i)$, the subsequence of the w most recent messages up to and including m_i . Successive windows overlap in $w-1$ messages: R_i and R_{i+1} share messages $m_{i-w+2}, \dots, m_i$.

We assume a passive adversary (a subscriber) who knows the mechanism family, topic hierarchy, window size w , and public publisher attributes. The adversary’s goals are: (1) link y to the true publisher p , (2) infer sensitive publisher attributes, or (3) deterministically know sensitive message payload content. Goals (1) and (2) constitute the publisher privacy problem; goal (3) is the message content privacy problem.

4 PROBLEM STATEMENT

Classic publish–subscribe architectures expose two classes of information that, taken together, strip publishers of both indistinguishability and plausible deniability.

Indistinguishability is violated at the publisher level. Each message carries an unmodified topic drawn from a fine-grained hierarchy, allowing a subscriber to deterministically attribute a message to its originating publisher. A privacy-preserving system must ensure that every delivered message is consistent with a distinct class of publishers, so that no individual publisher can be identified as the source.

Plausible deniability is violated at the message level. The broker forwards payloads verbatim, so a subscriber who observes a sequence of outputs can discover the exact data a publisher emitted

without any uncertainty. A privacy-preserving system should ensure quantitative privacy guarantees through modifying the output such that the output distribution under one payload sequence is statistically close to the output distribution under any neighboring sequence, giving the publisher plausible deniability over what was sent.

Existing work addresses these concerns only partially [1]. Static k -anonymity assumes a fixed dataset and does not account for publishers entering and leaving a sliding window [1]. w -event-level differential privacy protects a single message within a window w but ignores the reality of publish–subscribe architectures where naive aggregation creates batched output diminishing real-time utility; user-level privacy protects all messages but demands unbounded noise on infinite streams [1]. Most proposals treat the two guarantees in isolation, without specifying where each should be enforced or how they compose within the publish–subscribe architecture.

This paper defines a two-layer architecture in which publisher privacy (indistinguishability among k publishers) is enforced at the broker’s egress as output privacy, and content privacy (plausible deniability of payloads via w -event differential privacy) is enforced at the broker’s ingress as input privacy. We show that the w -event guarantee of Kellaris et al. [1] transfers to the per-message setting via a formal mapping, that the two layers compose without interference, and that the resulting system handles dynamic infinite streams while preserving per-message delivery and downstream utility, minimizing the privacy-utility trade-off.

4.1 Input Privacy versus Output Privacy

Before presenting each guarantee, we introduce a distinction that governs where and how privacy is enforced. In the differential-privacy literature, mechanisms are classified by the trust model as either local, where each data owner perturbs data before releasing it, or centralized, where a trusted curator collects raw data and perturbs the aggregate. In a publish–subscribe system, this distinction maps onto a spatial question: does the privacy transformation occur on the publisher side or on the subscriber side of the broker?

Definition 4.1 (Input Privacy). A mechanism provides input privacy if the privacy transformation is applied when the broker ingests the message, independently of any subscription. The guarantee holds for all downstream subscribers simultaneously.

Definition 4.2 (Output Privacy). A mechanism provides output privacy if the privacy transformation is applied at the broker’s egress, conditioned on the subscription that triggered delivery. The guarantee is tailored to each subscriber’s view.

Table 1 summarizes the mapping. Content privacy, hiding what the message contains, perturbs payloads on the publisher’s own stream as they enter the broker. The perturbation budget depends on which messages fall within the publisher’s rolling window and is agnostic of the subscriptions that will later receive those messages. Content privacy, therefore, operates as input privacy and follows the local trust model: each publisher’s stream is independently sanitized, and every subscriber who receives a given message sees the same perturbed payload.

Table 1: Input vs. output privacy in publish–subscribe.

	Content Privacy	Publisher Privacy
Protects	what m contains	who sent m
Trust model	local (input)	centralized (output)
Enforcement point	broker ingress	broker egress
Subscription-aware?	no	yes
Guarantee	w -event ϵ -DP	k -anonymity, l -div., t -close.
Output model	per-message (1-to-1)	per-message (1-to-1)
Technique	noise injection	topic generalization

Publisher privacy, hiding who sent a message, generalizes the message topic at delivery time. The generalization level required to achieve k -anonymity depends on which publishers appear in the subscriber’s view, which depends on the subscription’s topic subtree and the current window contents. Publisher privacy, therefore, operates as output privacy and follows the centralized trust model: the broker knows the publisher identities and must reason about each subscriber’s view to determine how far to generalize.

This dichotomy has a practical consequence: content privacy can be enforced once per publisher stream, globally, while publisher privacy must be enforced per subscriber (or per equivalence class of messages based on the topic hierarchy).

4.2 Message Content Privacy (Input Side)

Message content privacy protects sensitive information in the payload x , independently of publisher identity. Because the perturbation is calibrated to the privacy budget remaining in the publisher’s own rolling window, without reference to any subscription, this is an instance of input privacy (Definition 4.1).

The central challenge is to provide a meaningful privacy guarantee over an infinite stream while preserving per-message output: every input message must produce an output message. We build on the w -event differential privacy framework of Kellaris et al. [1], adapting its stream model and neighboring definition to the publish–subscribe setting. The goal is to preserve a particular publishers plausible deniability about whether or not a payload value reflects the actual value.

4.2.1 Adapting the w -Event DP Framework to Publisher Streams. Kellaris et al. [1] introduced w -event ϵ -differential privacy to bridge the gap between event-level privacy, which hides a single event but ignores temporal correlations, and user-level privacy, which hides all events of a user but requires unbounded noise on infinite streams. Their key insight is that sensitive information is typically revealed not by a single event in isolation but by a *sequence* of events occurring within w consecutive timestamps—for example, a commuter’s trajectory over w road segments.

We adapt this insight to the setting of *publisher plausible deniability* over message streams. In our formulation, a publisher emits an infinite stream of messages $S = (m_1, m_2, \dots)$, where each m_i denotes the message released at timestamp i . Each message is drawn from a public domain $\mathcal{X}$ and may carry content that, if linked to the publisher across several consecutive timestamps, could compromise the publisher’s deniability—for instance, a sequence of w successive

messages whose topics or stylistic features collectively identify the publisher. A single message in isolation may be innocuous, but a contiguous window of w messages can constitute a fingerprint that undermines plausible deniability.

The privacy mechanism $\mathcal{M}$ intercepts the raw stream and releases a perturbed transcript $(\tilde{m}_1, \tilde{m}_2, \dots)$. At each timestamp i , $\mathcal{M}$ observes m_i and outputs $\tilde{m}_i$, a modified version of the message whose perturbation is controlled by a privacy budget ϵ_i . To formalise the guarantee, we lift the notion of neighbouring stream prefixes from databases to individual messages.

Definition 4.3 (*w-Neighbouring Message Streams*). Two stream prefixes $S_t = (m_1, \dots, m_t)$ and $S'_t = (m'_1, \dots, m'_t)$ are *w-neighbouring* if:

- (i) for each position i where $m_i \neq m'_i$, the messages m_i and m'_i are *adjacent*, meaning they differ by the contribution of a single publisher (i.e. one stream includes the publisher's authentic message at i while the other includes a default or absent message), and
- (ii) for any two positions $i_1 < i_2$ where $m_{i_1} \neq m'_{i_1}$ and $m_{i_2} \neq m'_{i_2}$, it holds that $i_2 - i_1 + 1 \leq w$.

In words: the two streams are identical except possibly at positions that all fall within a contiguous window of w timestamps, and at each such position the streams differ by exactly one publisher's message.

Definition 4.4 (*w-Event ϵ -Differential Privacy for Publisher Streams*). A mechanism $\mathcal{M}$ that takes as input a stream prefix of arbitrary length and outputs a perturbed transcript satisfies *w-event ϵ -differential privacy* if, for all measurable sets O of transcripts, all *w-neighbouring* stream prefixes S_t, S'_t , and all t ,

$$\Pr[\mathcal{M}(S_t) \in O] \leq e^\epsilon \cdot \Pr[\mathcal{M}(S'_t) \in O].$$

The guarantee is one of plausible deniability: an adversary who observes the perturbed transcript cannot reliably distinguish whether a particular publisher authored any specific sequence of up to w consecutive messages. Setting $w = 1$ recovers event-level privacy, protecting only a single message; letting $w \rightarrow \infty$ approaches user-level privacy but demands unbounded perturbation. Choosing a finite w strikes a practical balance, it protects the publisher against adversaries who correlate messages across a bounded temporal window, while keeping the perturbation (and hence the utility loss) independent of the total stream length.

Following Kellaris et al., we realise *w-event* privacy through a sliding-window decomposition. The mechanism $\mathcal{M}$ is composed of per-timestamp sub-mechanisms $\mathcal{M}_1, \mathcal{M}_2, \dots$, each operating with independent randomness, where $\mathcal{M}_i$ processes m_i and is ϵ_i -differentially private. The *w-event* guarantee then reduces to the budget constraint

$$\forall i, \sum_{k=i-w+1}^i \epsilon_k \leq \epsilon, \quad (1)$$

i.e. the sum of the per-timestamp budgets within every sliding window of size w must not exceed the total budget ϵ . The challenge, and a focus of the mechanisms we build upon, lies in *dynamically allocating* these budgets so that messages requiring substantive perturbation receive adequate budget, while timestamps at which

the stream content is well-approximated by a prior release can be skipped, conserving budget for future use

Definition 4.5 (*w-Event ϵ -Differential Privacy* [1, Def. 4]). A mechanism $\mathcal{M}$ satisfies *w-event ϵ -differential privacy* if for all *w-neighbouring* stream prefixes S_t, S'_t , all output sets O , and all t :

$$\Pr[\mathcal{M}(S_t) \in O] \leq e^\epsilon \cdot \Pr[\mathcal{M}(S'_t) \in O].$$

The key operational tool is the sliding window methodology: decompose $\mathcal{M}$ into per-timestamp sub-mechanisms $\mathcal{M}_1, \mathcal{M}_2, \dots$, each using independent randomness and spending budget ϵ_i .

THEOREM 4.6 (SLIDING WINDOW BUDGET CONSTRAINT [1, THM. 3]). If $\mathcal{M}$ decomposes into sub-mechanisms $\mathcal{M}_1, \dots, \mathcal{M}_t$ with $\mathcal{M}_i(D_i) = o_i$, each $\mathcal{M}_i$ using independent randomness and satisfying ϵ_i -differential privacy, then $\mathcal{M}$ satisfies *w-event ϵ -DP* whenever

$$\forall i \in [t] : \sum_{k=i-w+1}^i \epsilon_k \leq \epsilon. \quad (2)$$

This constraint treats ϵ as a sliding budget: any rolling window of w consecutive positions may collectively spend at most ϵ . Mechanisms differ in how they allocate this budget, uniformly (Uniform), by periodic sampling (Sample), or adaptively via dissimilarity-driven schemes such as Budget Distribution (BD) and Budget Absorption (BA) [1].

4.2.2 Mapping Publish-Subscribe to the *w-Event Stream Model*. Kellaris et al.'s model assumes a curator who collects multi-row databases at each timestamp and publishes aggregate counts. Publish-subscribe is structurally different: messages arrive individually, each message has a single publisher, and subscribers expect per-message output. We now define a formal mapping that bridges this gap, preserving the *w-event* guarantee while respecting the one-message-in, one-message-out contract.

Stream construction. Let $\sigma = (m_1, m_2, m_3, \dots)$ be the infinite message stream ordered by arrival. We map each message position i to a timestamp in Kellaris's model by constructing the induced payload stream

$$S^\sigma = (D_1^\sigma, D_2^\sigma, D_3^\sigma, \dots) \text{ where } D_i^\sigma \triangleq \{x_i\}$$

is a singleton database containing only the payload of message m_i . Each position in the Kellaris stream corresponds to exactly one pub-sub message, and the broker produces exactly one output y_i per position. The rolling window $R_i = (m_{i-w+1}, \dots, m_i)$ corresponds to the active window of size w at timestamp i in Kellaris's framework.

Payload query and sensitivity. Let $\mathcal{X}_B \subset \mathcal{X}$ denote a bounded payload domain (e.g., sensor readings quantized to $[0, B]$). The identity query $Q(D_i^\sigma) = x_i$ has sensitivity

$$\Delta(Q) = \max_{x, x' \in \mathcal{X}_B} \|x - x'\|_1 = B.$$

If payloads are further quantized into histogram bins (e.g., the broker maps x_i to a d -dimensional indicator vector over predefined ranges), then $\Delta(Q) = 1$, exactly as in the original setting. We write Δ for the sensitivity of whatever per-message query the broker evaluates.

Content neighborhood. We adapt Definition ?? to single-message databases.

Definition 4.7 (w-Neighboring Message Streams). Two message-stream prefixes $\sigma_t = (m_1, \dots, m_t)$ and $\sigma'_t = (m'_1, \dots, m'_t)$ are w -content-neighboring if:

- (i) for each position i where $m_i \neq m'_i$, the two messages differ only in their payload: $p_i = p'_i$, $s_i = s'_i$, $\tau_i = \tau'_i$, but $x_i \neq x'_i$; and
- (ii) all positions where payloads differ fit within a contiguous window of w consecutive positions, i.e., for any $i_1 < i_2$ with $x_{i_1} \neq x'_{i_1}$ and $x_{i_2} \neq x'_{i_2}$, we have $i_2 - i_1 + 1 \leq w$.

This is a direct instantiation of Kellaris's w -neighboring (Definition ??) with the singleton databases $D_i^\sigma = \{x_i\}$ and $D_i^{\sigma'} = \{x'_i\}$ playing the role of neighboring databases. Condition (i) restricts the neighboring relationship to payload changes; publisher identity, topic, and timestamp are held fixed, because content privacy specifically protects what the message contains, not who sent it. Condition (ii) inherits the original requirement: all payload modifications are temporally localized within a window of w messages.

Intuitively, w -content-neighboring captures the following threat: an adversary observes the full output stream $(y_1, y_2, \dots)$ and tries to distinguish whether a particular publisher sent payload sequence $(x_{i-w+1}, \dots, x_i)$ or an alternative sequence $(x'_{i-w+1}, \dots, x'_i)$ within any w consecutive messages. The w -event guarantee ensures these two scenarios are statistically indistinguishable.

Why w and not a single event. If we set $w = 1$, the guarantee degenerates to event-level privacy [1]: each message's payload is individually protected, but correlated payloads across successive messages, such as a temperature ramp in a smart thermostat, or an accelerating heart-rate sequence, are exposed. By choosing $w > 1$, we protect entire payload sequences of length w , at the cost of distributing the privacy budget ϵ across more positions and therefore adding more noise per message.

4.2.3 Per-Message w -Event Privacy for Publish-Subscribe. We now state the content-privacy guarantee and show that Kellaris et al.'s Theorem 3 carries over to the per-message pub-sub setting.

Definition 4.8 (Per-Message w -Event Content Privacy). A publish-subscribe content-privacy mechanism $\mathcal{M}$ satisfies message w -event ϵ -differential privacy if for all w -content-neighboring message-stream prefixes σ_t, σ'_t (Definition 4.7), all output sets O , and all t :

$$\Pr[\mathcal{M}(\sigma_t) \in O] \leq e^\epsilon \cdot \Pr[\mathcal{M}(\sigma'_t) \in O].$$

The mechanism produces one output y_i per input message m_i .

THEOREM 4.9 (SLIDING BUDGET SUFFICIENCY FOR PUB-SUB). *Let $\mathcal{M}$ be decomposed into per-message sub-mechanisms $\mathcal{M}_1, \mathcal{M}_2, \dots$, where $\mathcal{M}_i$ takes message m_i as input, produces output $y_i = Q(m_i) + \text{Lap}(\Delta/\epsilon_i)$, and uses independent randomness. Then $\mathcal{M}$ satisfies per-message w -event ϵ -DP whenever*

$$\forall i: \sum_{k=i-w+1}^i \epsilon_k \leq \epsilon. \quad (3)$$

What changes, and what does not. The proof structure is identical to Kellaris et al.'s, the factorization over independent sub-mechanisms and the telescoping product over the active window

Table 2: Mapping between Kellaris et al.'s original stream model and the publish-subscribe instantiation. The algebraic structure of Theorem 4.9 is identical to [1, Thm. 3]; only the objects being manipulated differ.

Component	Kellaris et al.	Pub-Sub
Stream element $S[i]$	Database D_i (many rows)	Singleton $\{m_i\}$
Neighboring at i	Add/remove one row in D_i	Change payload x_i in m_i
Query Q	Column-count vector c_i	Per-message payload function
Sensitivity Δ	1 (binary matrix)	B (bounded payload domain)
Output o_i	Noisy count vector	Noisy per-message output y_i
Active window	w timestamps	w messages (rolling)
# outputs per position	1 (aggregate)	1 (per-message)

are unchanged. What changes is the interpretation of each component:

The crucial structural preservation is that each position i produces exactly one output using independent randomness, and the neighboring relation localizes all differences within w consecutive positions. These are the only two properties that Kellaris et al.'s proof requires; the internal structure of each database (multi-row vs. singleton windowed message with many attributes) and the nature of the query (aggregate count vs. per-message windowed aggregation payload function) affect only the noise scale Δ/ϵ_i , not the validity of the sliding-window argument.

Budget allocation strategies. All budget-allocation strategies from [1] transfer directly to the per-message setting. Under **Uniform**, every message receives budget $\epsilon_i = \epsilon/w$, yielding noise scale $\lambda = \Delta \cdot w/\epsilon$ per message. Under **Sample**, every w -th message receives the full budget $\epsilon_i = \epsilon$ (noise scale Δ/ϵ), and intervening messages are approximated by repeating the last published output. Under **BA** (Budget Absorption), the broker privately checks whether the incoming payload has changed significantly from the last release; if not, the message is skipped (approximated by the previous output), and its budget is absorbed by the next publication that does occur, reducing noise precisely when the data changes most. Schäler et al. [2] showed empirically that the stream's amplitude is the decisive factor in choosing among these strategies.

4.2.4 Structural Mismatches and Residual Costs. The mapping above resolves three of the five mismatches identified in earlier formulations of this problem:

- (1) **Unit of protection (resolved).** By constructing singleton databases $D_i^\sigma = \{x_i\}$, each message payload becomes the unit of protection. The window size w is chosen to reflect the publisher's temporal activity horizon, e.g., w consecutive heartbeats or w consecutive traffic reports, rather than a fixed database snapshot.
- (2) **Output granularity (resolved).** Each position produces exactly one output, matching the pub-sub contract. Multiple messages sharing a timestamp simply occupy consecutive positions in σ .

- (3) **Publisher privacy is absent (resolved by layering).** Publisher re-identification is handled by the output-side mechanism (Section 4.3), which operates independently and composes with content privacy.
- (4) **No aggregate query (partially resolved).** If the per-message query Q has bounded sensitivity Δ , the Laplace mechanism applies directly. For payloads with unbounded domains, the broker must either quantize the payload (bounding Δ) or switch to an aggregate-histogram output model, which sacrifices per-message delivery.
- (5) **Approximation semantics (inherent).** When a budget-allocation strategy, such as Sample or BA skips a message, the subscriber receives a repeat of the previous output. This is meaningful when consecutive payloads are correlated (e.g., slowly varying sensor readings) but can be misleading for bursty or heterogeneous publisher traffic. The choice of w and the allocation strategy must be tuned to the stream's dynamics [2].

PROPOSITION 4.10 (COMPOSABILITY OF MESSAGE-CONTENT PRIVACY). *If rewriting is applied independently per window, then $\epsilon_{\text{global}} = \sum_{t=1}^N \epsilon_t$.*

4.3 Publisher Privacy (Output Side)

Publisher privacy ensures that no rewritten message can be deterministically attributed to its originating publisher. It does not hide message content; it hides who sent it. Because the level of topic generalization required depends on which publishers appear in the subscriber's current window, which varies across subscriptions, publisher privacy is an instance of output privacy (Definition 4.2).

For publisher privacy, we group messages using the rolling window. Let $\text{pub}(m)$ return the publisher of m . For rewritten output y produced from some message in window R_i , define the consistent publisher set

$$P_i(y) \triangleq \{ \text{pub}(m) : m \in R_i, T(m) = y \}.$$

Definition 4.11 (Rolling k -Publisher Privacy). A mechanism satisfies rolling k -publisher privacy if $|P_i(y)| \geq k$ for every position i and every output y produced from a message in R_i .

When the topic space $\mathcal{S}$ forms a rooted tree, the broker enforces this by generalizing each message's topic, walking up the hierarchy until at least k distinct publishers share the rewritten topic within the current rolling window. Because the window slides one message at a time, the generalization level may change with every new arrival: a topic that was sufficiently general at position i may need further generalization at position $i+1$ if the oldest message in the window (which contributed a distinct publisher) slides out.

However, k -publisher privacy alone is insufficient if all k candidates share the same sensitive attribute. Let $A(m)$ denote a sensitive publisher attribute and $\text{Sens}_i(y) \triangleq \{A(m) : m \in R_i, T(m) = y\}$.

Definition 4.12 (Rolling l -Diversity). A mechanism satisfies rolling l -diversity if $|\text{Sens}_i(y)| \geq l$ for every i and y .

Definition 4.13 (Rolling t -Closeness). A mechanism satisfies rolling t -closeness if $d(\text{Dist}_i(y), \text{Dist}_i) \leq t$ for every y , where Dist_i is the global distribution of sensitive attributes across all messages in R_i .

Table 3: A stream of six messages. Attribute A is a sensitive publisher characteristic (fleet size). The rolling window of size $w=4$ at position 6 covers m_3-m_6 .

Pos.	Msg	Publ.	A	Topic	Payld.	τ
1	m_1	p_1	small	Elm/1st	12	08:00
2	m_2	p_2	large	Elm/2nd	27	08:02
3	m_3	p_3	small	Oak/1st	8	08:04
4	m_4	p_4	large	Oak/3rd	19	08:06
5	m_5	p_1	small	Elm/1st	13	08:08
6	m_6	p_2	large	Elm/2nd	26	08:10

Discrete attributes are generalized via the topic hierarchy; continuous attributes are quantized into bins and merged until all constraints hold. Because the rolling window overlaps, the broker must re-evaluate constraints whenever a new message enters or an old message exits the window.

PROPOSITION 4.14 (COMPOSABILITY OF PUBLISHER PRIVACY). *Under rolling windows, the global publisher-privacy level is $k_{\text{global}} = \min_i k_i$, where k_i is the publisher-privacy level achieved in window R_i .*

4.4 A Toy Example: Smart-City Traffic Monitoring

To make both guarantees concrete, we trace a small example through the entire pipeline. Consider a smart-city traffic service where four taxi companies (p_1-p_4) publish vehicle counts on road segments. Messages arrive one at a time; the broker uses a rolling window of size $w = 4$.

Table 3 shows a stream of six messages. Every message produces exactly one output, there is no aggregation into batches.

Content privacy (input side, rolling w -event DP). Content privacy is applied first, at the broker's ingress, on each publisher's stream independently. Under per-message w -event DP with $\epsilon = 1$ and $w = 4$, the broker perturbs each message's payload as it arrives, subject to the constraint that the total budget in any rolling window of four consecutive messages from the same publisher is at most ϵ . This perturbation is subscription-agnostic: every subscriber who later receives a given message sees the same noisy payload.

Table 4 traces the budget evolution for the six-message stream under two strategies. The Pos. column is the position in the stream; the rolling window at each position covers the four most recent messages.

Under **Uniform**, every message receives the same budget $\epsilon/w = 0.25$ and corresponding noise scale $\Delta/0.25 = 4$. The noise is moderate but constant.

Under **BA**, the broker notices that m_5 's payload (13) barely differs from m_1 's (12), both are Elm/1st readings, so it skips m_5 and repeats the previous Elm/1st output y_1 . The saved budget (0.25) becomes available. Similarly, m_6 (payload 26) is close to m_2 (payload 27), so it is also skipped. When the next genuinely different Elm message arrives, BA will absorb the accumulated budget, producing a much lower-noise output precisely when the data has changed. This is the core advantage of adaptive allocation [1]: budget is spent where it matters most.

Table 4: Content privacy under rolling w -event DP ($\epsilon=1$, $w=4$, $\Delta=1$). Uniform allocates $\epsilon/w = 0.25$ per message. BA skips messages whose payloads barely changed, absorbing budget for future use. $\sum_{k=i-3}^i \epsilon_k \leq 1$ holds for every $i \geq 4$.

Pos.	Payld.	Uniform		BA	
		ϵ_i	Output	ϵ_i	Output
1	12	.25	12+Lap(4)	.25	12+Lap(4)
2	27	.25	27+Lap(4)	.25	27+Lap(4)
3	8	.25	8+Lap(4)	.25	8+Lap(4)
4	19	.25	19+Lap(4)	.25	19+Lap(4)
5	13	.25	13+Lap(4)	0 (skip)	repeat y_1
6	26	.25	26+Lap(4)	0 (skip)	repeat y_2
Budget in [3, 6]:		1.00		0.50	
Budget in [4, ...]:		$\leq 1 \checkmark$		$\leq 1 \checkmark$	

Table 5: Publisher privacy under rolling windows ($k=2$, $l=2$). The broker re-evaluates as the window slides.

Window	Rewritten topic	P	Sens	Consistent publishers
R_4	Elm/*	2	2	p_1 (small), p_2 (large)
R_4	Oak/*	2	2	p_3 (small), p_4 (large)
R_5	Elm/*	2	2	p_2 (large), p_1 (small)
R_5	Oak/*	2	2	p_3 (small), p_4 (large)

Because the perturbation is applied per publisher stream at ingestion, before any subscription is matched, content privacy is **input privacy**: an Elm-street subscriber and an all-streets subscriber both receive the same noisy payload for m_1 .

Publisher privacy (output side, $k=2$). Publisher privacy is applied second, at delivery time, and its behavior depends on the subscriber's topic filter. At position 4, the rolling window $R_4 = (m_1, m_2, m_3, m_4)$ contains four publishers. A subscriber whose filter matches Oak/1st can immediately identify p_3 from the unmodified topic. To achieve 2-publisher privacy for this subscriber, the broker generalizes topics at egress: both m_3 and m_4 are delivered with topic Oak/*, so the subscriber sees two possible publishers. A different subscriber whose filter matches Elm/* sees m_1 and m_2 generalized to Elm/*, consistent with either p_1 or p_2 .

At position 5, the window slides to $R_5 = (m_2, m_3, m_4, m_5)$. Message m_1 has exited; m_5 has entered with topic Elm/1st from publisher p_1 . The broker must re-evaluate: within R_5 , Elm messages come from p_2 (via m_2) and p_1 (via m_5), so Elm/* still satisfies $k=2$ for a subscriber filtering on Elm. Table 5 shows the publisher-privacy state at two window positions.

Because the generalization level depends on which publishers appear in the subscriber's view, and different subscribers with different topic filters see different publisher sets, publisher privacy is output privacy.

Two layers compose. First, at ingress, the broker perturbs each message's payload using the rolling w -event budget (input side, content privacy). Then, at egress, the broker generalizes topics per

subscriber to achieve k -publisher privacy (output side, publisher privacy). Neither layer interferes with the other: payload perturbation does not alter publisher identities or topics, and topic generalization does not alter the already-perturbed payloads.

4.5 Utility Objective

Privacy enforcement necessarily distorts the data. We formalize utility as the fidelity between original and rewritten outputs, measured over rolling windows and globally.

4.5.1 Notation. Let $R_1, R_2, \dots$ be the sequence of rolling windows (each of size w , overlapping by $w-1$ messages). Let T_i denote the rewriting mechanism applied to message m_i . Let $y_i = T_i(m_i)$ be its rewritten output. Let $f : \mathcal{Y} \rightarrow \mathcal{Z}$ be a downstream task (e.g., a classifier or regressor) and $\ell : \mathcal{Z} \times \mathcal{Z} \rightarrow \mathbb{R}_{\geq 0}$ a loss function.

4.5.2 Per-Window Task Utility.

Definition 4.15 (Per-Window Task Loss). For the rolling window at position i , the per-window task loss is

$$L_i \triangleq \frac{1}{w} \sum_{k=i-w+1}^i \ell(f(y_k), f(m_k)).$$

The broker's optimization objective is to minimize this loss subject to both privacy constraints:

$$\min_{\{T_k\}} L_i \quad \text{s.t.} \quad \begin{cases} |P_i(y)| \geq k & \forall y \text{ in } R_i, \\ |\text{Sens}_i(y)| \geq l & \forall y \text{ in } R_i, \\ d(\text{Dist}_i(y), \text{Dist}_i) \leq t & \forall y \text{ in } R_i, \\ \sum_{k=i-w+1}^i \epsilon_k \leq \epsilon & (\text{content DP budget}). \end{cases} \quad (4)$$

4.5.3 Distributional Utility. When the downstream task is unknown, a task-agnostic fidelity measure is preferable. Let P_i denote the distribution of a feature vector $\phi(m)$ over original messages in R_i , and Q_i the distribution of $\phi(T(m))$ over rewritten messages.

Definition 4.16 (Per-Window Distributional Utility). The per-window distributional utility loss is

$$U_i \triangleq D_{\text{KL}}(P_i \parallel Q_i).$$

Lower U_i indicates higher fidelity.

4.5.4 Global Utility. Because rolling windows overlap, the global utility is a weighted average over message positions rather than over disjoint windows. Each message m_j participates in exactly $\min(j, w)$ rolling windows. The global utility is therefore:

$$U_{\text{global}} = \frac{1}{N} \sum_{i=w}^N U_i,$$

where N is the stream length observed so far. This can be computed incrementally: when position $i+1$ arrives, the broker updates the running average by adding U_{i+1} and removing the contribution of the window that has slid past a reporting horizon.

The composability of the utility measure carries over from the additive structure: because each U_i depends only on messages within R_i , the global average decomposes as a simple mean, enabling incremental monitoring, adaptive control of privacy parameters

Table 6: Two-layer privacy architecture for publish–subscribe with rolling windows.

	Content Privacy	Publisher Privacy
Protects	what m contains	who sent m
Privacy model	input (local)	output (centralized)
Subscription-aware	no	yes
Window model	rolling, size w	rolling, size w
Guarantee	w -event ϵ -DP	k -anon., l -div., t -close.
Output model	per-message (1-to-1)	per-message (1-to-1)
Enforcement	noise injection at ingress	topic generalization at egress
Budget constraint	$\sum_{k=i-w+1}^i \epsilon_k \leq \epsilon$	deterministic per window

(k, l, ϵ_i) , and principled budget allocation to minimize cumulative utility loss.

4.6 Summary of Privacy Architecture

Table 6 summarizes the two-layer architecture. Both layers use rolling windows of the same size w , and both produce per-message

output; every input message yields exactly one rewritten message. Content privacy is enforced first, at the broker’s ingress (input side), via noise injection on each publisher’s stream. Publisher privacy is enforced second, at the broker’s egress (output side), via topic generalization whose level is determined by the subscriber’s view.

The formal mapping of Section 4.2.2 ensures that the w -event guarantee of Kellaris et al. [1] transfers to the publish–subscribe setting without modification to the proof structure. What changes is the interpretation: singleton-message databases replace multi-row population databases, per-message payload queries replace aggregate count queries, and the window slides over message positions rather than timestamps.

REFERENCES

- [1] Georgios Kellaris, Stavros Papadopoulos, Xiaokui Xiao, and Dimitris Papadias. 2014. Differentially Private Event Sequences over Infinite Streams. *Proceedings of the VLDB Endowment* 7, 12 (2014), 1155–1166. <https://doi.org/10.14778/2732977.2732989>
- [2] Christine Schäler, Thomas Hütter, and Martin Schäler. 2023. Benchmarking the Utility of w -event Differential Privacy Mechanisms – When Baselines Become Mighty Competitors. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1830–1842. <https://doi.org/10.14778/3594512.3594515>